# ⚖ Adhikar AI

## Project Notes — Complete Guide

*A grounded AI assistant for Indian Labour Law Q&A*

| | |
|---|---|
| Topic | Indian Labour Law Q&A |
| Type | RAG-based AI Assistant |
| Backend | FastAPI (Python) |
| Frontend | React + Vite |
| AI Model | Google Gemini |
| Search | FAISS + BM25 (Hybrid) |
| Observability | LangSmith |
| Version | 1.0 |

# A little background!

*Why this project exists*

Millions of workers in India do not know their legal rights.

They don't know how much overtime pay they are owed. They don't know how many maternity leave days are guaranteed by law. They don't know if their employer can legally fire them.

Getting this information is hard:

1. Hiring a lawyer costs money most workers cannot afford.
2. Government websites have the law text, but it is written in complex legal language.
3. Asking HR or a coworker often gives the wrong answer.

Adhikar AI solves this by giving workers a conversational AI they can ask questions — and every answer comes with the exact source it was taken from.

# What is Adhikar AI

*Section 1 — Introduction*

> *Adhikar AI is an AI assistant that answers questions about Indian Labour Law. It always shows the exact source passages it used to build the answer, so users can verify every claim.*

The name "Adhikar" means "Rights" in Hindi. The project is built around one idea:

> *Workers deserve to know their rights — in plain language, with proof.*

### It supports all the major use cases

4. Workers asking about wages, overtime, and leave
5. Union representatives understanding unfair labour practices
6. Compliance officers checking statutory requirements
7. Anyone who needs a quick, cited answer about Indian labour law

### Key properties

- Grounded — answers only come from retrieved source passages, not from AI guesswork
- Cited — every response includes the documents it used
- Failure-aware — when it cannot find good sources, it says so instead of making something up
- Traceable — every request is logged and observable via LangSmith

> **Recap**
> Adhikar AI is a grounded legal Q&A assistant for Indian Labour Law.
> Every answer is tied to a source. If sources are weak, the system says so.

# Why do we need Adhikar AI

*Section 2 — Problem Statement*

### The problem with general AI tools

If you ask ChatGPT or Gemini a legal question, they often give a confident-sounding answer that is wrong. They might cite a law section that does not exist, or mix up rules from different states, or give outdated information.

> *The dangerous part is not when AI says 'I don't know.' The dangerous part is when it confidently gives a wrong legal answer — and someone acts on it.*

### Example queries that general AI gets wrong

8. "What is the overtime rate in Maharashtra factories?"
9. "How many days of maternity leave is a contract worker entitled to?"
10. "Can my employer legally deduct pay for going to the bathroom?"

For all of these, a general AI can give plausible-sounding but factually wrong answers. Adhikar AI solves this by grounding every answer in actual retrieved source text.

### The problem with legal websites

- They have the law text, but no conversational interface
- They cannot reason across multiple statutes for a combined question
- They do not explain things in plain language

### What Adhikar AI solves

| Problem | How Adhikar AI solves it |
|---|---|
| Legal text is hard to read | Explains in plain language (ELI5 mode) |
| AI tools hallucinate | Only answers from retrieved source passages |
| Lawyers are expensive | Free, available 24/7 |
| Hard to find multi-statute answers | Searches across the full indexed corpus |
| Cannot verify AI answers | Shows exact source passages with every response |

> **Recap**
> General AI tools hallucinate legal answers with high confidence — that is dangerous.

> Adhikar AI grounds every response in retrieved source text to prevent this.

# What This Document Covers

*Section 3 — Outline*

11. What is Adhikar AI and why it exists
12. How the system works end-to-end (step by step)
13. The tech stack — what tools are used and why
14. Every feature explained in detail
15. How the system handles failures and errors
16. Architecture — how all the pieces connect
17. Security decisions
18. Honest limitations (what it cannot do yet)
19. What the next version would look like

### My focus

- Clarity over complexity — every concept is explained from scratch
- Real behavior — descriptions match what the code actually does
- Honest trade-offs — no overclaiming
- The 80% approach — cover what matters most, deeply

# What Adhikar AI can do

*Section 4 — Features Overview*

> *Adhikar AI is a RAG-based Q&A system. RAG stands for Retrieval-Augmented Generation — it means the AI retrieves relevant documents first, then uses them to generate a grounded answer.*

## Core features

20. Answer labour law questions with cited sources
21. Stream responses live as they are generated (like ChatGPT typing)
22. Show a confidence score for every answer
23. Let users upload their own PDF documents as extra context
24. Evaluate answers for hallucination risk
25. Email answers to yourself or others
26. Switch between explanation styles — Lawyer, Worker, or Simple (ELI5)

## Who is it for?

| User | How they use it |
|---|---|
| Factory worker | "How many days of earned leave do I get?" |
| Domestic worker | "Can my employer not pay me for a public holiday?" |
| Union representative | "What counts as an unfair labour practice?" |
| Compliance officer | "Cite the section for minimum wage calculation in Rajasthan" |
| Paralegal | "What is the notice period required for retrenchment?" |

> **Recap**
> Adhikar AI can answer, cite, stream, evaluate, and email labour law answers.
> It works for workers, union reps, and compliance professionals alike.

# How Adhikar AI works

*Section 5 — End-to-End Flow*

When you ask a question, Adhikar AI follows these steps:

### Step 1 — You ask a question

> *Example: "I worked 54 hours last week in a Mumbai textile factory. What overtime pay am I entitled to?"*

### Step 2 — The system searches for relevant passages

It runs two searches at the same time and then combines the results.

- Dense search (FAISS) — finds passages that are semantically similar in meaning
- Keyword search (BM25) — finds passages that contain the exact words you used
- Fusion — combines both lists using a scoring method called Reciprocal Rank Fusion

Why two searches? Because meaning-based search is great for understanding what you meant, but keyword search is essential for finding exact statute numbers like "Section 59" or "Code on Wages 2019".

### Step 3 — Reranking

The top results from Step 2 are re-scored by a second, more powerful model called a cross-encoder.

- The cross-encoder reads the question and each passage together — not separately
- This makes it much better at deciding which passage actually answers your question
- If the reranker fails to load (e.g. low memory), the system skips this step gracefully and uses the Step 2 order

### Step 4 — Building the answer

The top passages are assembled into a "context block" and given to Google Gemini along with your question.

The prompt tells Gemini:

27. Here are the source passages — only answer from these
28. If they do not contain enough information, say so explicitly — do not guess
29. Use the tone the user asked for (lawyer / worker / simple)

### Step 5 — Streaming the response

Gemini generates the answer token by token. You see words appear on screen as they are written, just like ChatGPT.

- You can pause, resume, or stop the stream at any time
- If Gemini fails mid-stream, the system sends a structured error — it does not leave you with a broken screen

### Step 6 — You see the final answer

The final answer includes:

30. The answer text
31. Source passages that were used
32. A confidence score (how well the sources matched your question)
33. Status: complete, partial, or error

### Optional Step 7 — Evaluate

You can click "Evaluate" to get a quick check:

- Does the answer actually use words from the question?
- Are the cited sources consistent with what was said?
- Are there any claims in the answer that no source supports?

### Optional Step 8 — Email

You can email the answer to yourself. The system uses Gmail OAuth2 or SMTP and includes security checks to prevent abuse.

> **Recap**
> Query → Hybrid Retrieval → Reranking → Prompt Building → Gemini Generation → Stream to User
> Every step has a fallback if something goes wrong.

# Adhikar AI Components

*Section 6 — System Architecture*

> *Adhikar AI is built from modular components. Each component has one job, has defined failure modes, and can be replaced or upgraded independently.*

## The main components

| Component | What it does |
|---|---|
| Frontend (React) | The user interface — chat, settings, email, evaluation |
| Backend Orchestrator | The brain — coordinates all other components per request |
| Retrieval Pipeline | Searches the statute corpus using FAISS + BM25 |
| Reranker | Re-scores retrieved passages for answer relevance |
| LLM Layer (Gemini) | Generates the grounded answer |
| Streaming System (SSE) | Sends the answer to the browser word by word |
| Email System | Sends the answer via Gmail OAuth2 or SMTP |
| Observability (LangSmith) | Records every request for debugging and analysis |

# Frontend

*Section 6.1 — Component Detail*

> *The frontend is a React single-page application built with Vite. It handles the chat interface, streaming display, settings, upload, evaluation, and email.*

### What it does

- Sends queries to the backend
- Connects to the backend via SSE (Server-Sent Events) to receive streamed tokens
- Renders answer text word-by-word as it arrives
- Lets you pause, resume, or stop the stream
- Shows source passages and confidence score
- Has a Settings page where you enter your Gemini API key
- Has sliders for top_k (number of sources to retrieve) and alpha (keyword vs meaning balance)

### Failure mode

If the SSE connection drops mid-stream (e.g. network error), the browser fires an error event. The UI transitions to an error state and shows whatever partial answer was received, with an error badge.

### Design note

The API key is stored on the client side in development. This is intentional for demo convenience. A production deployment must move key management to the server side.

# Backend Orchestrator

*Section 6.2 — Component Detail*

> *The orchestrator is created fresh for every request. It receives the query and all parameters, then calls Retrieval → Reranking → Generation in sequence.*

### Why create it fresh per request?

A shared object would mean one user's request could accidentally affect another user's. Creating it fresh avoids any shared state bugs.

### What it owns

- The SSE stream connection
- The responsibility to always send a final payload — whether generation succeeded, failed, or was stopped
- Error classification — it decides if a Gemini failure is an auth error, a quota error, or a timeout

### Model caching

Gemini model instances are expensive to create. The orchestrator uses a cache, but the cache key is:

> *cache key = (model_name, hash_of_api_key)*

This ensures that if you change your API key, the old cached model (tied to the old key) is not reused. The raw API key never appears in any log.

# Retrieval Pipeline

*Section 6.3 — Component Detail*

> *The retrieval pipeline finds the most relevant passages from the statute corpus for a given query. It uses two search methods and combines their results.*

### Dense search — FAISS

FAISS converts your query into a vector (a list of numbers that represents its meaning) and finds the corpus passages whose vectors are closest.

- Good at: finding passages that match the meaning of your question
- Limitation: may miss passages that contain specific legal terms your question used
- Runs on CPU — loaded into memory at backend startup

### Sparse search — BM25

BM25 is a keyword-based search algorithm. It counts how often your query words appear in each passage.

- Good at: finding exact matches for statute names, section numbers, legal terms
- Limitation: does not understand meaning — "wages" and "salary" are different words to BM25
- Pure Python — fast enough for the current corpus size

### Fusion

Both searches return a ranked list. The pipeline combines them with a formula:

> *score = alpha × dense_rank_score + (1 - alpha) × keyword_rank_score  alpha = 0.5 (default) → equal weight to both alpha = 1.0 → meaning-only alpha = 0.0 → keyword-only*

### Deduplication

Both searches can return the same passage. Duplicates are removed before the list is sent to the reranker — otherwise the reranker wastes time on the same text twice.

> **Recap**
> FAISS finds passages by meaning. BM25 finds passages by keywords.
> Both lists are combined, duplicates removed, then sent to the reranker.

# Reranker

*Section 6.4 — Component Detail*

> *The reranker is a cross-encoder model that re-scores the retrieved passages. It looks at your question and each passage together to decide which ones actually answer your question best.*

## Why is reranking needed?

FAISS and BM25 score passages independently from the question — they don't really understand if the passage answers the question or just talks about the same topic.

Example:

- Passage A: talks about overtime in the Shops and Establishments Act
- Passage B: talks about overtime in the Factories Act
- Both passages have similar embeddings — FAISS may rank them equally
- The cross-encoder reads your question ("factory worker overtime") and Passage B together, and gives Passage B a much higher score

## Cross-encoder vs Bi-encoder

| Bi-encoder (FAISS) | Cross-encoder (Reranker) |
|---|---|
| Encodes query and passage separately | Reads query + passage together |
| Fast — runs on pre-built index | Slower — runs fresh for every (query, passage) pair |
| Good for recall (find the right topics) | Good for precision (find the right answer) |
| Can handle millions of passages | Practical for top 5–20 candidates only |

## Failure mode

If the cross-encoder model fails to load (e.g. not enough memory), the system skips reranking. The answer quality drops slightly but the system keeps working.

# LLM Layer — Google Gemini

*Section 6.5 — Component Detail*

> *The LLM layer sends the assembled context and query to Google Gemini and streams the response back through the orchestrator.*

### Error classification

Not all Gemini failures are the same. The system classifies them:

| Error type | Cause | Message shown to user |
|---|---|---|
| Auth failure | Invalid or revoked API key | "API key is invalid" |
| Quota exceeded | Free tier rate limit hit | "Rate limit reached — try again later" |
| Timeout | Gemini took too long to respond | "Provider timeout — try again" |
| Provider outage | Gemini service is down | "Provider error — try again later" |

Before this classification was built, quota errors appeared to users as invalid key errors. This confused users who kept re-entering their (valid) key. Fixing the classification required parsing the Gemini error body, not just the HTTP status code.

# Streaming System

*Section 6.6 — Component Detail*

> *Streaming is how words appear on your screen as they are generated, instead of waiting for the full answer. Adhikar AI uses SSE (Server-Sent Events) for this.*

### How SSE works

34. The browser opens a connection to /chat/stream
35. The backend keeps the connection open and sends small "events" as tokens are generated
36. The browser's EventSource API receives each event and renders the token
37. At the end, one final "final_payload" event is sent with the complete answer, sources, confidence, and status
38. The connection closes

### The final payload guarantee

> *No matter what happens — success, failure, or user stop — the stream always ends with exactly one final_payload event. The frontend always has a complete state to render.*

### What can go wrong

- Provider fails mid-stream → system catches the exception, sends a final_payload with status=error and whatever partial text was received
- User clicks Stop → frontend disconnects EventSource, backend detects the closed connection and stops calling Gemini (no wasted quota)
- Long stream gets cut by a proxy timeout → system sends a keepalive comment every 15 seconds to prevent this

# Email System

*Section 6.7 — Component Detail*

> *The email system lets users send an answer to themselves or a specified address. It supports two transport methods: Gmail OAuth2 (preferred) and SMTP with an app password (simpler fallback).*

### Gmail OAuth2 flow

39. A refresh token is generated once during setup using the Google OAuth consent screen
40. For each email send, the refresh token is exchanged for a short-lived access token
41. The access token is used to call the Gmail API

### SMTP fallback

42. Uses Python's built-in smtplib
43. Requires a Gmail App Password (not your Google account password)
44. Simpler to set up for development

### Security controls

| Threat | What the system does |
|---|---|
| Header injection | Rejects any sender input containing \r or \n characters |
| HTML injection in body | All answer text is HTML-escaped before inserting into the email template |
| Partial config | If neither Gmail nor SMTP is fully configured, the request fails with a clear error — it does not attempt a partial send |

### Real incident — OAuth refresh token mismatch

During development, the email system kept failing with "unauthorized_client" even though a refresh token existed.

Root cause: A refresh token is tied to the exact client_id and client_secret it was generated with. Using a token with different credentials silently fails. The fix was to regenerate the refresh token with the correct credentials.

# Observability — LangSmith

*Section 6.8 — Component Detail*

> *LangSmith is an observability platform for LLM applications. Adhikar AI sends a trace for every chat request to LangSmith so you can inspect exactly what happened.*

### What a trace contains

- The query that was sent
- The passages that were retrieved
- The prompt that was built
- The full answer that was generated
- How long each step took
- Any errors that occurred

### Why observability matters in LLM systems

Unlike regular software, LLM applications have non-deterministic outputs. Two identical queries can produce slightly different answers. Without tracing, debugging a bad answer means guessing — with tracing, you can see exactly which passages were retrieved, what prompt was used, and why the answer came out a certain way.

### Configuration

```
# In .env
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=your_key_here
LANGSMITH_PROJECT=AdhikarAI
LANGSMITH_ENDPOINT=https://api.smith.langchain.com

# If LANGSMITH_TRACING=false — zero impact on performance
```

> **Recap**
> LangSmith traces every request: inputs, outputs, timings, errors.
> Tracing is optional — disable it and the system runs identically without it.

# Tech Stack

*Section 7 — Tools used and why*

> *Every tool in Adhikar AI was chosen for a specific reason. Here is each tool, what it does, why it was chosen, and what trade-off was accepted.*

| Tool | What it does | Why chosen |
|---|---|---|
| FastAPI | Backend web framework | Native async support + SSE streaming built in. Much less overhead than Django for an API-only backend. |
| Google Gemini | Generates the answers | Free tier available for demo. Has a streaming API and supports long contexts. |
| FAISS (CPU) | Dense vector search | Fast nearest-neighbour search on CPU. No external service needed. Index is rebuildable. |
| BM25 (rank_bm25) | Keyword search | Pure Python. Critical for matching exact statute section numbers and legal terminology. |
| Cross-encoder | Reranks retrieved passages | Jointly scores (query, passage) pairs — much better at identifying the actual answer than embedding similarity alone. |
| React + Vite | Frontend UI | EventSource API handles SSE natively. Component-based for easy state management. |
| LangSmith | Observability / tracing | Chain-level request tracing. Captures inputs, outputs, and timings per request. |
| Gmail OAuth2 + SMTP | Email transport | OAuth2 is the secure path. SMTP with app password is the easy dev fallback. |

**Recap**

All tools were chosen to work on a regular laptop without GPU.

Every tool has a fallback or graceful degradation path.

# The Confidence Score

*Section 8 — How confidence works*

> *The confidence score is a number between 65% and 93% that tells you roughly how well the retrieved sources matched your question. It is NOT a probability that the answer is correct.*

### What goes into it

45. How relevant the top passage was (cross-encoder score)
46. How many passages were retrieved above a relevance threshold
47. How many of your query words appear in the source passages

### Why is it clamped to 65%–93%?

The raw signal can swing wildly. A very out-of-domain query might give a raw score of 20%. A near-exact statute lookup might give 98%. Showing these raw numbers creates two problems:

48. 20% makes users ignore a correct answer from a genuinely strong retrieval
49. 98% implies a statistical certainty that the system has not earned

> *The clamp range of 65%–93% communicates 'reasonably grounded' to 'well grounded' without implying calibrated probability.*

### Limitations

- It is heuristic — weights were set manually, not trained on data
- There is no labelled dataset to validate it against
- A 78% score does not mean 78% probability of being correct — it means retrieval was fairly strong

### Future plan

To properly calibrate this, the project needs a labelled dataset: (question, answer, is_answer_correct). With that, a technique called Platt Scaling can map the raw signal to real probabilities. Without the dataset, this is not possible.

> **Recap**
> Confidence = heuristic signal based on retrieval quality.
> Clamped to 65%–93% to avoid misleading users with extreme values.
> Not a statistical probability. Future calibration needs labelled data.

# Prompt Engineering

*Section 9 — How prompts are built*

> *A prompt is the full instruction sent to Gemini. Adhikar AI builds this prompt carefully to prevent hallucination and to produce answers in the right style.*

### Prompt structure (in order)

50. Role — "You are a legal information assistant for Indian labour law"
51. Source passages — the retrieved text, each tagged with its source
52. Anti-hallucination instruction — "If the context does not contain a clear answer, say so. Do not answer from general knowledge."
53. Style modifier — changes tone based on the selected mode
54. The user's query

### Why this order?

The anti-hallucination instruction comes after the source passages — this forces it to operate against the actual retrieved content, not in the abstract. The style modifier comes last so it shapes output style without interfering with the grounding instruction.

### The three explanation modes

| Mode | What it does |
|---|---|
| Lawyer | Uses statute names and section numbers. Assumes legal background. Precise citations. |
| Worker | Plain language. Step-by-step. Focuses on what to do, not legal theory. |
| ELI5 (Simple) | Explains like talking to someone with no legal background. Uses everyday analogies. |

All three modes receive the same retrieved passages. The only difference is the style instruction added to the prompt.

> **Recap**
> Prompts are built in 5 layers: role → sources → anti-hallucination → style → query.
> Three explanation modes (Lawyer, Worker, ELI5) change only the output style, not the sources.

# Challenges & Problems Faced

*Section 10 — Real incidents from development*

> *This section documents real problems that happened during development. It is written like incident notes — what went wrong, why, what was fixed.*

### 1. GPU hang → OS crash → full rebuild

During a heavy session (indexing + reranking + running the IDE all at once), the machine froze under memory pressure. A forced shutdown during active disk writes caused filesystem corruption. The machine booted to an error screen with no storage visible.

- Lost: the environment, all local indexes, all installed packages
- Fix: Clean OS reinstall. Treated indexes as rebuildable outputs, not precious files. Documented setup steps properly.
- Lesson: Run compute-heavy offline tasks separately from the interactive UI if possible

### 2. Gemini API key marked invalid when it was actually valid

Valid API keys were sometimes flagged as invalid. Users kept re-entering their correct key.

- Root cause 1: Model instances were cached by model name only. After a key change, the old instance (with the old key) was reused.
- Root cause 2: Quota errors and auth errors were both treated as "invalid key" because both return HTTP 4xx.
- Fix: Cache key is now (model_name, hash_of_api_key). Error classification now reads the response body to distinguish quota from auth failures.

### 3. Gmail OAuth — "unauthorized_client" error

Email sending failed even though a refresh token was present.

- Root cause: The refresh token was generated with a different OAuth client ID and secret than what was in the config. Gmail silently rejects this.
- Fix: Regenerate the refresh token with the correct client credentials. Test the full OAuth flow (get token → refresh → send) before deploying.

### 4. .env config not loading in some setups

The backend reported Gmail OAuth was missing even though the .env file had all the values.

- Root cause: .env loading used a relative path. Starting the backend from a different directory caused it to not find the file.
- Fix: Load .env from a fixed project-root path. Much more predictable.

### 5. Streaming left users in a broken state on provider failure

When Gemini failed mid-stream, the SSE connection closed without a final payload. The frontend had no complete state object and showed a blank or partial screen.

- Fix: The orchestrator now catches all exceptions during streaming and always sends a final_payload event with status=error before closing the connection.

### 6. Header injection risk in email

The email endpoint accepted a user-supplied sender address. Email headers are newline-sensitive — a \r or \n character in the sender value would terminate the current header and allow injecting new ones (including Bcc fields).

- Fix: Validate the sender input with a regex that rejects \r and \n characters. Return HTTP 400 if found.

### 7. Confidence score swinging too much

The raw confidence signal could reach below 30% for out-of-domain queries and above 95% for exact matches. Showing raw values confused users.

- Fix: Clamp the displayed confidence to the band 65%–93%.

> **Recap**
> Real problems encountered: GPU crash, OAuth mismatch, streaming failures, header injection, .env loading issues.
> Every problem above has a documented fix in the codebase.

# What makes this production-grade

*Section 11 — Comparison*

> *Most student projects work in the happy path. Production-grade systems handle the unhappy path just as well — failures, edge cases, security, observability.*

| Dimension | Typical project → Adhikar AI |
|---|---|
| Failures | try/except + print() → Classified errors: auth vs quota vs timeout, each handled differently |
| Streaming | Not implemented → SSE with pause/resume/stop + guaranteed final payload |
| Retrieval | Single vector search → FAISS + BM25 + fusion + deduplication + reranking |
| Confidence | Not tracked → Heuristic signal, clamped, shown in UI with honest limitations |
| Observability | Console logs → LangSmith traces: inputs, outputs, timings, errors per request |
| Security | Not considered → Header injection check, HTML escaping, fail-closed email config |
| Model caching | New instance every request → Cache keyed by (model_name, api_key_hash) |
| Setup | Works on my machine → Deterministic .env loading, health endpoint, pinned deps |
| Failures documented | Not tracked → Real incidents written up with root cause + fix |

> **Recap**
> This project handles failures explicitly, is fully observable, and has security controls.
> The failure engineering section documents real incidents from development — not hypothetical ones.

# Honest Limitations

*Section 12 — What this system cannot do yet*

> *A project that is honest about its gaps is more trustworthy than one that overclaims. Here is what Adhikar AI does not do yet.*

### No user authentication

Right now, anyone who finds the URL can use the system. There is no login, no accounts, no rate limiting per user. You cannot deploy this publicly as-is.

### No GPU acceleration

Embedding, reranking, and indexing all run on CPU. This works fine for a demo, but it would be too slow for many users at once. A GPU instance would make the cross-encoder 5–10x faster.

### Confidence is not statistically calibrated

The confidence score is a heuristic. It has not been validated against a dataset of questions with known correct answers. You cannot trust the number as a probability.

### Limited corpus

The document index covers a curated selection of Indian labour statutes. It does not comprehensively cover all state amendments. It does not automatically update when laws change.

### No CI/CD or deployment pipeline

This runs as a localhost development setup. There is no Docker container, no automated testing pipeline, no staged rollout process.

### Prompt injection not fully defended

A user could craft a query that tries to override the system prompt ("Ignore previous instructions and..."). The system has no explicit defence against this beyond the LLM's own resistance.

> **Recap**
> No auth. No GPU. No calibrated confidence. No deployment pipeline.
> These are known gaps, not hidden ones — and they have clear paths to fix.

# Future Upgrades

*Section 13 — What the next version would look like*

### 1. Authentication + Rate Limiting

- Add JWT-based login (Google OAuth or email/OTP)
- Per-user and per-IP rate limiting at the API gateway
- Move API key management fully server-side

### 2. Better Infrastructure

- Replace FAISS in-memory with a managed vector database (Pinecone, Weaviate, or pgvector) — enables horizontal scaling and real-time updates
- GPU instance for embedding + reranking — 5–10x latency improvement
- Separate indexing service — so corpus updates don't restart the serving backend

### 3. Calibrated Confidence

- Build a labelled Q&A dataset: (question, answer, is_correct)
- Use Platt Scaling to map the raw confidence signal to real probabilities
- Display actual error bars, not just a single number

### 4. Better Evaluation

- Automated evaluation on every corpus update using ROUGE-L and BERTScore
- Citation accuracy check: does the cited passage actually contain the answer?
- Human expert review pipeline for quality assurance

### 5. Deployment + CI/CD

- Containerise with Docker
- GitHub Actions: lint + unit tests + integration tests on every push
- Staged rollout for model and prompt changes before full deployment

### 6. Hallucination Reduction

- Retrieve → Generate → Verify loop: after generating an answer, re-retrieve to confirm source support
- Reject answers where a significant fraction of claims have no matching source
- Fine-tune Gemini on the labour law domain

> **Recap**
> Priority order: Auth → Better infra → Calibration → CI/CD → Hallucination reduction.
> All upgrades have a clear path from the current codebase.

# Final Recap

*Everything in one place*

| Topic | Summary |
|---|---|
| What it is | A grounded AI assistant for Indian Labour Law. Every answer cites its sources. |
| Who it's for | Workers, union reps, compliance officers, paralegals |
| How it works | Query → Hybrid Retrieval (FAISS+BM25) → Reranking → Gemini → Streamed Response |
| Confidence score | Heuristic signal clamped to 65–93%. Not a probability. Honestly labelled. |
| Streaming | SSE-based. Always ends with a final_payload. Pause/resume/stop supported. |
| Email | Gmail OAuth2 + SMTP fallback. Header injection and HTML injection protected. |
| Observability | LangSmith traces every request. Optional. Zero impact when disabled. |
| Failures handled | Auth vs quota vs timeout classified separately. GPU crash documented. OAuth mismatch fixed. |
| Limitations | No auth, no GPU, no calibrated confidence, no deployment pipeline — all known and documented. |
| Next version | Auth + managed vector DB + GPU + calibrated confidence + CI/CD |